
Titulo del TFG
Title in english



Trabajo de Fin de Grado
Curso 2025–2026

Autor
Eva Paula Mik

Director
Ismael Sagredo Olivenza

Colaborador
Colaborador no definido. Usa \colaboradorPortada

Grado en Desarrollo de Videojuegos
Facultad de Informática
Universidad Complutense de Madrid

Titulo del TFG
Title in english

Trabajo de Fin de Grado en Desarrollo de Videojuegos

Autor
Eva Paula Mik

Director
Ismael Sagredo Olivenza

Colaborador
Colaborador no definido. Usa \colaboradorPortada

Convocatoria: *Junio 2026*

Grado en Desarrollo de Videojuegos
Facultad de Informática
Universidad Complutense de Madrid

16 de abril de 2026

Dedicatoria

*A toda la gente que nos ha hecho llegar hasta
aquí,
un besazo.
A todos los que han estado a nuestro lado.*

Agradecimientos

Os amo!

Mik

Os amo!

Pau

Os amo!

Eva

Resumen

Titulo del TFG

Para ello primero se buscaron datasets públicos de gestos, no encontrándose ninguno que se adaptara a las necesidades del estudio. Por ello se creó una herramienta para traducir animaciones de Mixamo a [CSVs](#) para que pudieran ser entendidas por los modelos a entrenar. Esta herramienta generó un gran número de animaciones pero muy desbalanceadas en el número de animaciones de cada tipo.

Para solucionar el problema del desbalance, se realizó una serie de pruebas con usuarios (N=65) en las cuales se les pidió que realizaran 3 tomas de cada gesto mientras llevaban el traje de captura de movimiento puesto. Esta recolección de datos resultó en un total de 975 animaciones, habiendo un total de 195 gestos humanos de correr, saludar, señalar, pegar y sentarse. Se omitió el gesto de baile en las pruebas por el desbalanceo en el dataset generado por ordenador. Esta omisión sirvió para equilibrar más el número de animaciones, pero no para terminar del todo con el desbalanceo en los datos estandarizados.

Una vez se tuvo el dataset, se implementaron distintos modelos de [IA](#) para la detección de gestos bajo una interfaz web para su facilidad de uso. Estos modelos fueron: [LSTM](#), [CNN](#), [RNN](#) y [Random Forest](#). Tras realizar los entrenamientos de todos los modelos, se observó que el modelo [Random Forest](#) era con diferencia el que mejores datos obtenía, con un 95 % de precisión en entrenamiento y un 86 % en test. Los resultados de los modelos basados en redes neuronales fueron bastante menores, pudiendo ser esto así por la falta de datos, de tiempo de entrenamiento y de hardware suficiente para explotar mejor los modelos.

Palabras clave

Captura de movimiento, Unity, Inteligencia Artificial, Comunicación no verbal, Perception Neuron, TensorFlow, YDF, Keras, Realidad Virtual

Abstract

Title in english

To achieve this, public gesture datasets were first sought, but none were found that met the study's needs. Therefore, a tool was created to convert Mixamo animations into [CSVs](#) format so they could be understood by the models to be trained. This tool generated a large number of animations, but they were highly imbalanced in the number of animations for each type.

To address the imbalance issue, a series of user trials (N=65) were conducted, where participants were asked to perform three instances of each gesture while wearing the motion capture suit. This data collection resulted in a total of 975 animations, with 195 human gestures for running, greeting, pointing, punching, and sitting. The gesture of dancing was omitted from the trials due to the imbalance in the dataset generated by computer animations. This omission helped balance the number of animations but did not completely eliminate the imbalance in the standardized data.

Once the dataset was established, various [AI](#) models for gesture detection were implemented under a web interface for ease of use. These models included: [LSTM](#), [CNN](#), [RNN](#), and [Random Forest](#). After training all the models, it was observed that the [Random Forest](#) model significantly outperformed the others, achieving 95% accuracy in training and 86% in testing. The results from the neural network-based models were considerably lower, likely due to insufficient data, limited training time, and hardware constraints that hindered better model performance.

Keywords

Motion capture, Unity, Artificial Intelligence, Non-verbal communication, Perception Neuron, TensorFlow, YDF, Keras, Virtual Reality

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	1
1.3. Plan de trabajo	1
2. Estado de la Cuestión	3
2.1. Simulación física en videojuegos	3
2.1.1. Motores físicos	4
2.1.2. Motores de videojuegos	4
2.1.3. Simulación de telas	4
2.1.3.1. ¿Qué es Cloth?	4
2.1.3.2. El problema de optimización	5
2.1.3.3. Soluciones	5
3. Descripción del Trabajo	7
3.1. Búsqueda de un dataset	7
3.1.1. Dataset de la Universidad Carnegie Mellon	7
3.1.2. Kaggle	8
3.2. Aplicación final	9
3.2.1. Funcionamiento de la aplicación	10
4. Conclusiones y Trabajo Futuro	13
4.1. Conclusiones	13
4.1.1. Conclusiones de la búsqueda de datasets	13
4.1.2. Conclusiones de la blablabla	13
4.1.3. Limitaciones	13
4.2. Trabajo Futuro	13
Contribuciones Personales	15
Bibliografía	17

A. Carteles para la generación del dataset	19
B. Formulario de recogida de citas	23
C. Resultados de los distintos modelos CNN	25
C.1. Intervalo 0.2s	25
C.2. Intervalo 0.4s	27
C.3. Intervalo 0.6s	28
C.4. Intervalo 0.8s	30
C.5. Intervalo 1.0s	31
Glosario	33
Licencia de uso del documento	35
Licencia de uso del código fuente	37

Índice de figuras

1.1. Diagrama de Gantt del proyecto	2
2.1. Imágen del traje de captura de movimiento XSens	5
3.1. Imágen del traje de captura de movimiento utilizado para la base de datos de la Universidad Carnegie Mellon (izquierda) y el esqueleto resultante (derecha). Fuente: https://mocap.cs.cmu.edu/info.php	8
3.2. Captura de la aplicación final desde el editor de Unity	10
3.3. Diagrama de flujo de la aplicación de demostración	12
A.1. Cartel colgado en la facultad de informática	19
A.2. Cartel colgado en redes sociales	20
A.3. Cartel colgado en pantallas de la facultad	21
B.1. Formulario de recogida de citas (primera parte)	23
B.2. Formulario de recogida de citas (segunda parte)	24
C.1. Esquema del modelo CNN con 0.2s de intervalo	25
C.2. Matriz de confusión del modelo CNN con 0.2s de intervalo	26
C.3. Gráfico de entrenamiento del modelo CNN con 0.2s de intervalo (me- jor val_accuracy = 0.28777)	26
C.4. Esquema del modelo CNN con 0.4s de intervalo	27
C.5. Matriz de confusión del modelo CNN con 0.4s de intervalo	27
C.6. Gráfico de entrenamiento del modelo CNN con 0.4s de intervalo (me- jor val_accuracy = 0.49473)	28
C.7. Esquema del modelo CNN con 0.6s de intervalo	28
C.8. Matriz de confusión del modelo CNN con 0.6s de intervalo	29
C.9. Gráfico de entrenamiento del modelo CNN con 0.6s de intervalo (me- jor val_accuracy = 0.44952)	29
C.10. Esquema del modelo CNN con 0.8s de intervalo	30
C.11. Matriz de confusión del modelo CNN con 0.8s de intervalo	30
C.12. Gráfico de entrenamiento del modelo CNN con 0.8s de intervalo (me- jor val_accuracy = 0.40583)	31
C.13. Esquema del modelo CNN con 1.0s de intervalo	31

C.14. Matriz de confusión del modelo CNN con 1.0s de intervalo	32
C.15. Gráfico de entrenamiento del modelo CNN con 1.0s de intervalo (mejor val_accuracy = 0.56002)	32

Índice de tablas

3.1. Reacción del npc al gesto predicho del jugador	11
---	----

Capítulo 1

Introducción

En los últimos años...

1.1. Motivación

Como se ha mencionado...

1.2. Objetivos

El objetivo general:

1. Extracción automática de las animaciones mencionadas anteriormente de bancos de animaciones y posterior procesamiento.
2. Extracción de las animaciones mediante la captura de movimiento de un grupo heterogéneo de usuarios.
3. Implementación, entrenamiento y comparación de distintos modelos de [IA](#), tanto redes neuronales (LSTM, CNN y RNN) como modelos de clasificación clásicos (Random Forest)
4. Implementación de una aplicación final a modo de demostración para enseñar los resultados en [VR](#) para aumentar el grado de inmersión.

Como gran objetivo secundario y debido a la falta de datasets de animaciones se ha propuesto publicar los datasets resultantes, tanto de las animaciones artificiales como de las capturas de usuarios.

1.3. Plan de trabajo

El plan de trabajo consiste en varios pasos:

1. Búsqueda de un dataset: generar un dataset lo suficientemente grande con varios ejemplos de gestos como para poder entrenar de forma adecuada diferentes modelos. Este dataset debe estar compuesto en su mayoría por datos de usuarios reales para poder identificar gestos lo más naturales posibles.
2. Implementación de modelos de IA: implementación de varios modelos de IA para poder hacer una comparativa entre ellos y decidir cuál es el más adecuado teniendo en cuenta su velocidad de predicción y su precisión. La implementación de dichos modelos debe contener: entrenador, forma de sacar las estadísticas del modelo final, búsqueda de hiperparámetros, generador de matriz de confusión y debe ser acoplable a una interfaz de usuario común a todos los entrenamientos.
3. Desarrollo de una interfaz de usuario para el entrenamiento y monitorización de los distintos modelos: desarrollo de una interfaz web para que los usuarios con menos experiencia en programación e IA puedan entrenar modelos capaces de predecir los gestos que necesiten sin necesidad de modificar el código ni tener que saber como funciona internamente la base de código.
4. Desarrollo de una aplicación final: desarrollo de una aplicación para las Oculus Quest a forma de demo en la que se conecte al modelo elegido y se pueda ver en tiempo real su uso.

El desarrollo de las tareas y su planificación se puede ver en la figura 1.1.

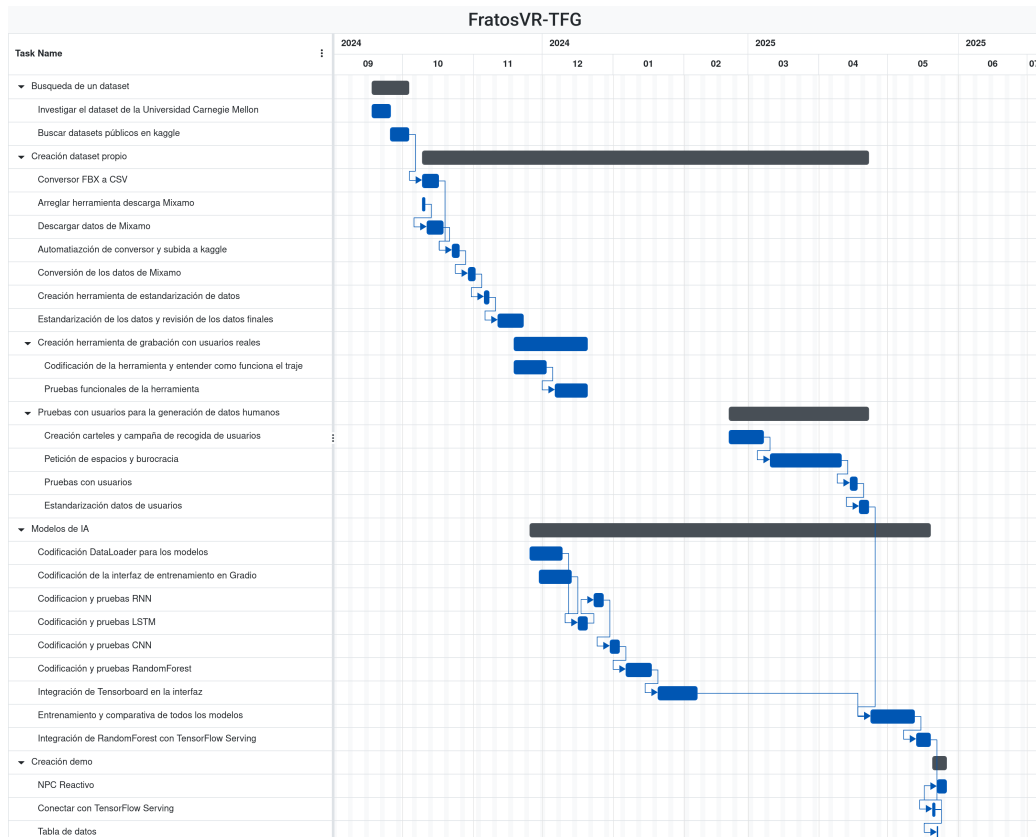


Figura 1.1: Diagrama de Gantt del proyecto

Capítulo 2

Estado de la Cuestión

En este capítulo se presenta una breve descripción del desarrollo de las simulaciones físicas en videojuegos, así como el hardware y software que ha hecho que esta evolución sea posible, se presenta la simulación de telas en específico, las maneras que existen de implementarla, los problemas de optimización que estas suponen y los avances en modelos de IA que permiten imaginar otras soluciones.

2.1. Simulación física en videojuegos

Basta con remontarse a la creación del Pong en 1972, para darse cuenta de la importancia que las simulaciones físicas han tenido en el ámbito de los videojuegos desde el primer momento. Al fin y al cabo, las físicas permiten crear situaciones de juego que no están predefinidas, enriqueciendo así la experiencia del jugador.

En sus inicios hubo muchas maneras de implementar estas físicas: prerenderización arrows

De todas formas, estos métodos carecen de COHERENCIA, la cual es imprescindible para la inmersión del jugador. Con la introducción del 3D, subió el nivel. La coherencia necesaria para la inmersión creció exponencialmente, lo que se busca ahora (hiperrealismo tal) y la libertad que se le quiere dar al jugador en la experiencia de juego, por lo que se le ha dau gran importancia a las cosas real-time.

Esto no nos sorprende, el mismo Ian Millingtong menciona en su libro Game Physics Engine Development [Millington \(2007\)](#) el juego Exile, creado por Peter Irvin y Jeremy Smith. Si bien este juego fue publicado en 1988 ya contaba con colisiones con intercambio de impulso"(esto no se entiende igual?), gravedad, fuerzas como por ejemplo el viento, explosiones, fuego y agua. Es probablemente el primer juego que contaba con un motor de físicas completo, y posteriormente se crearon muchos más y Estandar.incluir página web en la q el propio Peter explica esto y dice q Jeremy se muere? <https://inventivity.co.uk/exileami/about.htm> Dramatismo. Intensidad narrativa a la memoria.

2.1.1. Motores físicos

PHYSIS HAVOC BULLET intereses por trabajar en la CPU vs GPU

2.1.2. Motores de videojuegos

Breve definicion d lo q es un motor? Estos, se dividen en componentes y vamos a prestar atención a la evolución del componente encargado de la computación de calculos físicos.

hoy en día unreal y UNITY (justificamos por qué unity: libre acceso, motor principal usado para desarrollo?)

Unity es un motor de videojuegos creado por Unity Technologies en 2005. Su motor físico: PhysX, creado por la empresa Nvidia Corporation. Unreal es un motor de videojuegos creado por Epic Games en 1998 con el lanzamiento de Unreal Engine 1. La versión estable actual es Unreal Engine 5.5, lanzada en noviembre de 2024. El código fuente de este motor está escrito en C++ y es accesible desde Github ¹. Usa PhysX igualmente así q txau.

2.1.3. Simulación de telas

Los distintos tipos de simulaciones que se implementan en los motores de físicas previamente mencionados, podrían agruparse en los siguientes grupos: Collision detection, ragdoll physics, particle systems (fluids y fuzzy), deformable bodies (entre ellos rigid-bodies y soft-bodies). En concreto soft-bodies / cloth. Árnason (2008)

2.1.3.1. ¿Qué es Cloth?

Entendemos los objetos deformables o soft-bodies como aquellos que no son rígidos. Así como en los objetos rígidos podemos simular su movimiento sabiendo su motion y centro de masa, para los objetos deformables necesitamos tener en cuenta todas las partículas y propiedades físicas que lo componen. Entre estos encontramos elementos como el cloth, el agua o el viento Kenny Erleben (2005).

Cloth es un componente que nos proporciona Unity3D que funciona junto al Skinned Mesh Renderer para simular telas. Está basado en Cloth de Physics pero fue incorporado con el objeto de poder disponer de ropa para personajes. Tiene múltiples propiedades como la resistencia a la flexión, rigidez, damping, uso de gravedad. Cada cloth está formado por un conjunto de vértices (partículas) que tienen cierta libertad de movimiento dependiendo de sus constraints. Un constraint de distancia máxima 0 significa que está fijo en el espacio y que no se calculan sus deformaciones. La cloth cuenta con capacidad de colisionar internamente y con dos tipos de colliders externos: cápsulas o esferas. Objetos más complejos con los que se quiera colisionar tendrán que ser un conjunto de estos colliders.

Se trata la tela como una malla de partículas conectadas por restricciones. El Skinned Renderer calcula las transformaciones basándose en una malla de baja re-

¹Enlace a la página de Github de Epic: <https://github.com/epicgames>

solución sobre la que realiza los movimientos en los vértices, que dinámicamente se aplican en la malla real y obtenemos la simulación visual.

Un solver es un algoritmo computacional que se encarga de predecir el comportamiento del objeto deformable computer graphics y simulaciones físicas. Encontramos infinitos tipos de solvers. Los primeros solvers se basan en los algoritmos que emplean la fuerza bruta: realizan cálculos en masa para sistemas de muelles (MSS). Los solvers más actuales resultan en el PBD proceso iterativo que proyecta las posiciones de las partículas sobre un espacio de restricciones/constraints.

El solver de PhysX se basa en PBD: Position Based Dynamics desarrollado por Müller [Müller et al. \(2007\)](#). Este sistema omite las velocidades a la hora de resolver transformaciones, y trabaja directamente sobre las posiciones. XPBD [Macklin et al. \(2016\)](#) es una versión extendida que puede incorporarse en proyectos.

2.1.3.2. El problema de optimización

La simulación de cloth en tiempo real es un problema muy común, que aún sigue buscando optimizarse a día de hoy. En Unity, los frames decaen inmediatamente en cuanto se complica o aumenta la geometría de la tela. Impide el realismo y en juegos donde se necesita altas frame rates como los juegos en VR, supone un impedimento.

Mientras que el Cloth de PhysX puede delegar la simulación a CUDA capable GPUs, el Cloth implementado en Unity realiza todo su trabajo en la CPU, lo cual ralentiza mucho la ejecución.

2.1.3.3. Soluciones

Encontramos varios tipos de soluciones para la optimización de la simulación de telas:

- Optimización por CPU: encontramos la extensión de Obi Cloth [Method \(2019\)](#), que aprovecha el sistema de Jobs de Unity para paralelizar la simulación y acelerar la simulación sin salir de la CPU. Es uno de los paquetes más populares para solucionar este problema.
- Paralelización en GPU
- Entrenamiento con IA

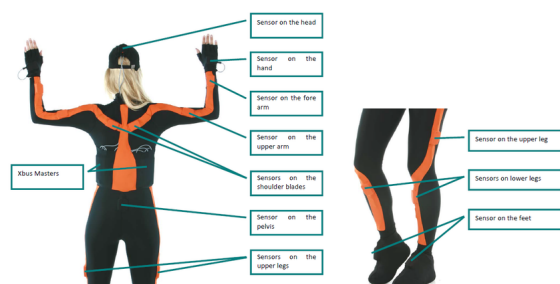


Figura 2.1: Imagen del traje de captura de movimiento XSens ²

²Fuente de la imagen:

<https://www.researchgate.net/figure/>

Capítulo 3

Descripción del Trabajo

*“In the distant future.
In a land abandoned by man.
Machines wage a continuous war.
Unaware of the futility of their actions.”*
— Nier Automata

3.1. Búsqueda de un dataset

Para poder entrenar modelos de IA se requiere una gran cantidad de datos, en este caso de animaciones de los gestos escogidos que se han mencionado en la introducción. Además, estos datos tienen que estar en un formato en el que se pueda extraer la información de los huesos (posición y rotación) para que pueda ser usado por los modelos, y tenían que ser compatibles con los huesos del traje de captura de movimiento.

3.1.1. Dataset de la Universidad Carnegie Mellon

La Universidad privada Carnegie Mellon, ubicada en Pittsburgh, Pensilvania, tiene disponible de forma gratuita un dataset de movimientos recogidos mediante la captura de movimiento.¹ Sin embargo este dataset no ha sido posible utilizarlo por tres motivos.

El primer motivo es la incompatibilidad del esqueleto utilizado por ellos con el utilizado por el traje de captura de movimiento Perception Neuron 3. Como se puede ver en ? su traje contiene 41 puntos usados para la captura, mostrados en la figura 3.1, dando a lugar al esqueleto mostrado en la misma figura.

¹Enlace a la página del dataset de la Universidad Carnegie Mellon: <https://mocap.cs.cmu.edu/>

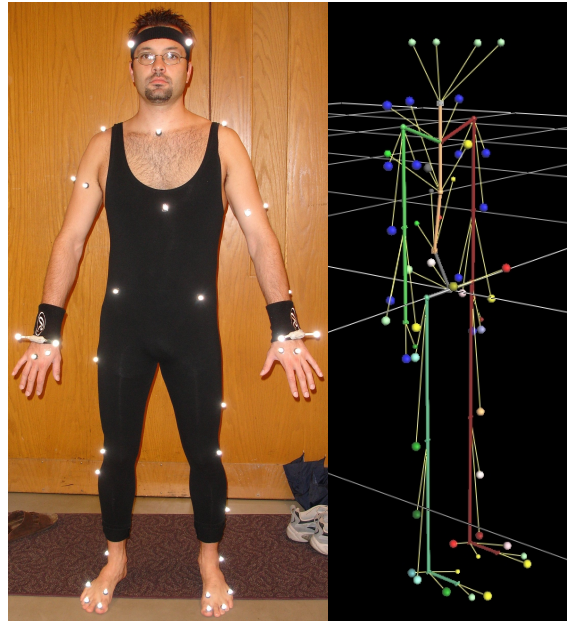


Figura 3.1: Imágen del traje de captura de movimiento utilizado para la base de datos de la Universidad Carnegie Mellon (izquierda) y el esqueleto resultante (derecha). Fuente: <https://mocap.cs.cmu.edu/info.php>

Por otro lado el traje utilizado en este estudio, el Perception Neuron 3, contiene 17 puntos, por lo que debido a las grandes diferencias de los esqueletos decidimos no utilizarlos.

El segundo motivo por el que no se ha usado el dataset de la Universidad Carnegie Mellon es el formato de las animaciones disponibles. Las animaciones presentes en el dataset vienen en tres formatos distintos: c3d, asf (o amc) y vsk (o v). Los formatos c3d, vsk y v son unos formatos en binario usados para objetos en 3D, por lo que no se puede usar de forma sencilla para extraer la información del propio archivo. Por otro lado los archivos asf y amc son archivos en formato ASCII con la información de los huesos. Sin embargo la documentación aportada para poder usar la información es incompleta.

El último motivo por el que no se ha usado este dataset es por la escasez de animaciones de los gestos buscados. Animaciones como “correr” sí están presentes (aunque no en abundancia), pero otras como “pelear” o “saludar” no.

Este último problema también ha estado presente en páginas específicas de bancos de datasets como Kaggle.

3.1.2. Kaggle

Kaggle es una página web dedicada a la ciencia de datos y el aprendizaje automático y la IA. En Kaggle se pueden encontrar datasets, modelos, código y competiciones de IA. Kaggle tiene una gran comunidad de usuarios y una gran selección de datasets para distintas finalidades.

En un primer momento, se pensó en usar datasets de imágenes/vídeo, así que se buscó conjuntos de datos de este formato. La búsqueda no obtuvo los resultados

esperados, ya que no se encontró datos de poses generales como las requeridas, y en concreto no se consiguió mucha cantidad de datos.

Tras analizar las desventajas de usar imágenes y vídeos, se pensó que era mejor usar un conjunto de datos basado en animaciones (o coordenadas de las mismas). Algunas de estas desventajas consideradas fueron:

- Complejidad de renderizado: las imágenes y/o vídeos requerirían de renderizar una cámara externa dentro del motor para capturar la presencia del usuario. Esto ya requeriría más potencia de computo, sin contar lo que requeriría el modelo.
- Transferencias de datos más grandes: al plantear un servidor para el modelo, se pensó en el tamaño de las imágenes que habría que mandar para hacer las inferencias. En el caso de tener una red de velocidad limitada sería mejor mandar los puntos concretos del traje que mandar imágenes completas. Las imágenes, incluso al ser de baja calidad, requerirían una cantidad muy grande de píxeles para hacer el modelo del usuario fácilmente reconocible.
- Pérdida de contexto: el traje de captura de movimiento te permite obtener datos tridimensionales de cada punto por defecto, sin necesidad de tener que recrear un esqueleto 3D con capas extra en el modelo.

Tras este cambio de enfoque, se buscó datasets de animaciones y modelos 3D. Resultó ser una búsqueda incluso peor que la anterior, ya que por la limitación de gente que tiene acceso a trajes de captura de movimiento, no había datasets públicos en dataset con este tipo de datos.

Ya que no se encontró un gran dataset que cumpliera con nuestros requerimientos se tomó la decisión de buscar en un banco de animaciones los gestos requeridos y transformar esas animaciones en un formato que pudiesen ser procesados.

3.2. Aplicación final

Una vez los modelos estaban preparados era necesaria una aplicación a modo de demostración que se pueda ejecutar en las gafas de [VR](#) y con el traje de captura de movimiento. La aplicación consta de un panel en el que introduces la IP de la máquina que tiene el servidor de Tensor ejecutándose con el modelo y la aplicación de Axis Studio para poder utilizar el traje. Además, tiene un panel de texto en el que aparecen los resultados de la predicción y un NPC que reacciona a tus gestos. En la figura [3.2](#) se puede ver una captura de esa aplicación.



Figura 3.2: Captura de la aplicación final desde el editor de Unity

3.2.1. Funcionamiento de la aplicación

Al introducir la IP en el cuadro de texto la aplicación se conecta con el traje y el servidor de Tensor. La conexión con el traje es la misma que la del apartado ??, mientras que para el servidor se utiliza una API REST mediante UnityWebRequest,² una API de Unity para comunicarse con servidores web. Esta ejecución se hace mediante una corrutina, la cual al principio espera la cantidad de tiempo óptima entre frame y frame que se determinó en el entrenamiento del modelo usado (en nuestro caso el Random Forest con 0.8 segundos).

La información de los huesos se guarda en una cola, donde se guarda la información actual y, una vez enviado al servidor, se elimina la más antigua, siendo siempre el tamaño de la cola de máximo dos elementos. Al pasar estos segundos la aplicación manda al servidor la información de los huesos en el frame actual y en el frame de hace 0.8 segundos (previamente guardado) en formato JSON mediante POST con protocolo HTTP, a lo que el servidor le responde con dos listas en formato JSON: una con los gestos y otra con el score que ha conseguido cada gesto en la predicción, coincidiendo el índice del score de un gesto con el índice del gesto de la anterior lista. Para hacer todas las conversiones entre cadenas de texto y el formato JSON se ha usado la clase JsonUtilty nativa en Unity.

Una vez se tenga la respuesta del servidor empieza de nuevo la corrutina, se ordena de mayor a menor score las predicciones mediante bubble sort para escribirlas en el panel correspondiente y se ejecuta la animación de respuesta del npc. Las animaciones de respuesta que tiene a los gestos se puede ver en la tabla 3.1

²Página de la documentación de Unity: <https://docs.unity3d.com/ScriptReference/Networking.UnityWebRequest.html>

Gesto	Animación de reacción
Bailar	Aplaudir
Saludar	Saludar
Señalar	Mirar
Sentarse	Sentarse
Pelear	Pose defensiva
Correr	Correr

Tabla 3.1: Reacción del npc al gesto predicho del jugador

Si hay un error de conexión en mitad de la ejecución se deberá ingresar de nuevo la IP de la máquina host en el campo de texto para reintentarlo.

En la figura [3.3](#) se puede ver el diagrama de flujo de esta aplicación.

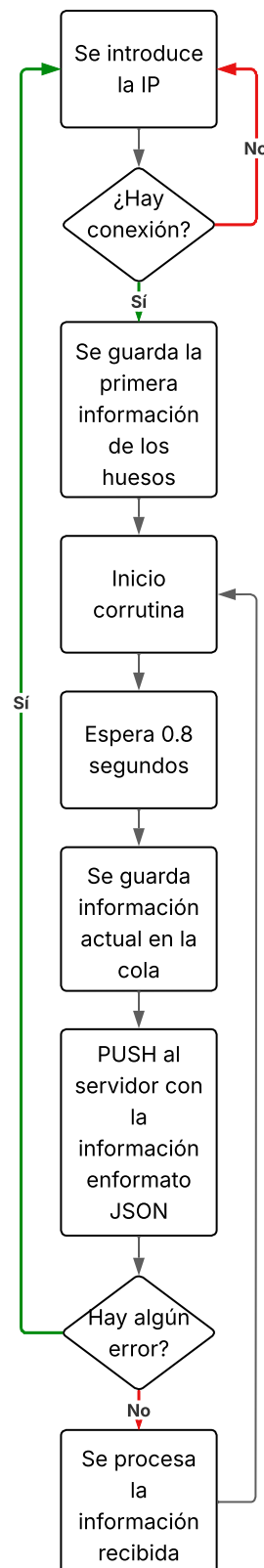


Figura 3.3: Diagrama de flujo de la aplicación de demostración

Conclusiones y Trabajo Futuro

4.1. Conclusiones

Las conclusiones de cada una de las partes del trabajo se presentan en las siguientes secciones de manera más detalladas.

4.1.1. Conclusiones de la búsqueda de datasets

Bla.

4.1.2. Conclusiones de la blablabla

Bla.

4.1.3. Limitaciones

Este estudio ha tenido una serie de limitaciones, desde blablabla Los modelos de redes neuronales no han dado los resultados esperados, esto puede ser por la falta de datos, que aunque haya un número que a simple vista parece suficiente no lo es para entrenar redes neuronales, por las limitaciones de hardware para entrenar modelos más complejos y por los tiempos requeridos para la realización de este estudio no ser lo suficientemente largos como para llevar a cabo entrenamientos más extensos y con mayor búsqueda de hiperparámetros.

Todas estas limitaciones, han llevado al siguiente plan de trabajo a futuro.

4.2. Trabajo Futuro

El trabajo a futuro de este estudio se puede centrar en varios esfuerzos:

- bla
- bla

- bla

Contribuciones Personales

Eva

Muchas cosas.

Pau

Muchas cosas.

Mik

Muchas cosas.

Bibliografía

The sciences, each straining in its own direction, have hitherto harmed us little; but some day the piecing together of dissociated knowledge will open up such terrifying vistas of reality, and of our frightful position therein, that we shall either go mad from the revelation or flee from the deadly light into the peace and safety of a new dark age.

H.P. Lovecraft, *The Call of Cthulhu*

ÁRNASON, B. Evolution of physics in video games. 2008.

KENNY ERLEBEN, K. H., JON SPORRING. *Physics-based Animation*. Charles River Media, 2005.

MACKLIN, M., MÜLLER, M. y CHENTANEZ, N. XPBD: position-based simulation of compliant constrained dynamics. En *Proceedings of the 9th International Conference on Motion in Games*, páginas 49–54. ACM, Burlingame California, 2016. ISBN 978-1-4503-4592-7.

METHOD, V. Obi cloth. <https://assetstore.unity.com/packages/tools/physics/obi-cloth-81333?aid=1011134eQ>, 2019. Accessed: (16/04/2026).

MILLINGTON, I. *Game Physics Engine Development*. CRC Press, 2007. ISBN 9781482267327.

MÜLLER, M., HEIDELBERGER, B., HENNIX, M. y RATCLIFF, J. Position based dynamics. *Journal of Visual Communication and Image Representation*, vol. 18(2), páginas 109–118, 2007. ISSN 10473203.

Carteles para la generación del dataset



Figura A.1: Cartel colgado en la facultad de informática

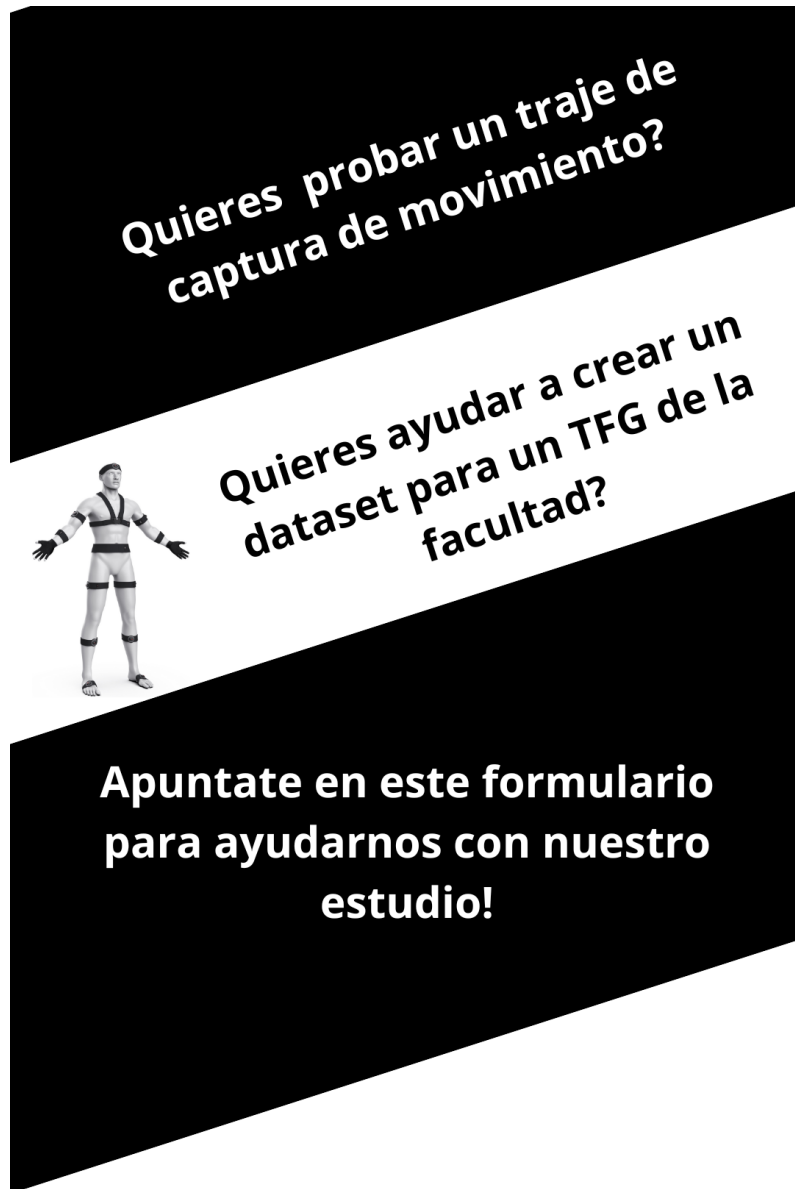


Figura A.2: Cartel colgado en redes sociales



Figura A.3: Cartel colgado en pantallas de la facultad

Formulario de recogida de citas

Participación Dataset para detección de poses

Necesitamos datos para un TFG de la Facultad de Informática de la UCM. Lo importante, **¿qué datos recopilamos?** La siguiente lista de datos son los datos que vamos a recopilar y hacer públicos en el propio estudio del tfg o en un dataset público en [kaggle](#):

- Edad
- Sexo
- Nacionalidad
- Datos de coordenadas de distintas partes del cuerpo dadas por un traje de captura de movimiento

Estos datos se recogerán en un formulario a parte el día de la prueba, siendo hechos desde un correo de los organizadores y permaneciendo totalmente anónimos sin correlación a la persona que ha realizado la prueba.

OBJETIVO DE LOS DATOS: los datos de edad, nacionalidad y sexo son meramente estadísticos, solo se usarán para demostrar la heterogeneidad (o falta de) de los datos tomados. Los datos de coordenadas se usarán para entrenar distintos modelos de clasificación de pose con la intención de poder decir con acierto el gesto o pose que está haciendo una persona mientras lleva el traje puesto.

LOCALIZACIÓN DE LA PRUEBA: [Facultad de Informática \(UCM\)](#), Calle del Prof. José García Santesmases, 9, Moncloa - Aravaca, 28040 Madrid

alejba02@ucm.es [Cambiar de cuenta](#)

* Indica que la pregunta es obligatoria

Correo *

Tu dirección de correo electrónico 

¿Aceptas la recogida de datos y la participación en el muestreo? *

☐ Si

☐ No

Siguiente

Página 1 de 2

Borrar formulario

Este formulario se creó en Universidad Complutense de Madrid.
¿Parece sospechoso este formulario? [Informe](#)



Figura B.1: Formulario de recogida de citas (primera parte)

Participación Dataset para detección de poses

alejba02@ucm.es [Cambiar de cuenta](#)

Horario disponible

A continuación marca las horas en las podrías estar disponible. Con la respuesta que proporciones te mandaremos un correo con un día y hora determinados.

Lunes

- ☐ 9:00-10:00
- ☐ 10:00-11:00
- ☐ 11:00-12:00
- ☐ 12:00-13:00
- ☐ 13:00-14:00
- ☐ 14:00-15:00
- ☐ 15:00-16:00
- ☐ 16:00-17:00
- ☐ 17:00-18:00
- ☐ 18:00-19:00
- ☐ 19:00-20:00

Figura B.2: Formulario de recogida de citas (segunda parte)

Resultados de los distintos modelos CNN

La línea verde en los gráficos de tensorboard indican la mejor combinación de hiperparámetros encontrada en cada caso

C.1. Intervalo 0.2s

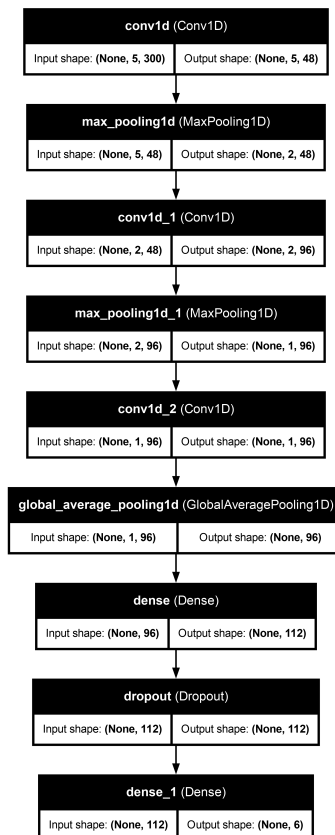


Figura C.1: Esquema del modelo CNN con 0.2s de intervalo

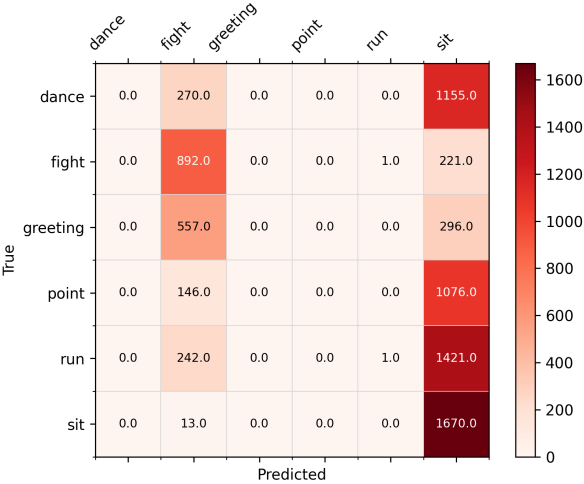


Figura C.2: Matriz de confusión del modelo CNN con 0.2s de intervalo

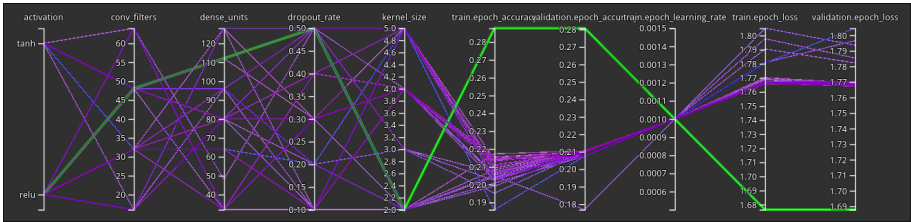


Figura C.3: Gráfico de entrenamiento del modelo CNN con 0.2s de intervalo (mejor val_accuracy = 0.28777)

C.2. Intervalo 0.4s

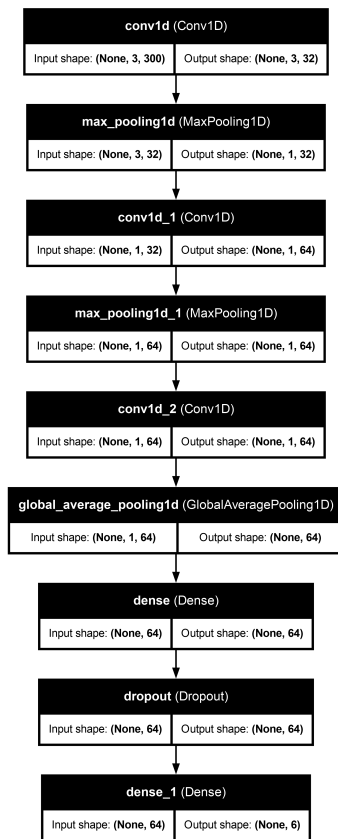


Figura C.4: Esquema del modelo CNN con 0.4s de intervalo

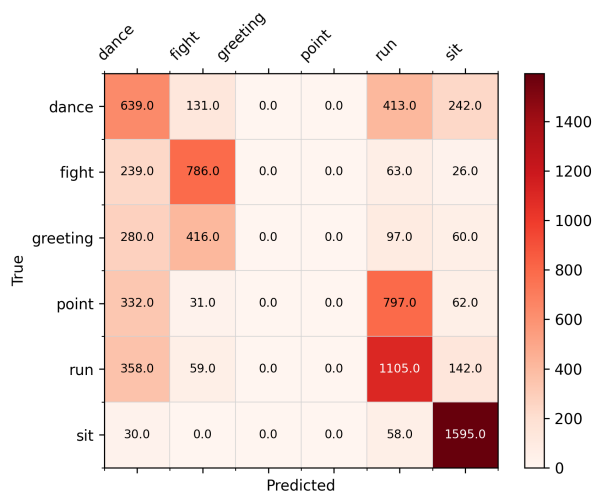


Figura C.5: Matriz de confusión del modelo CNN con 0.4s de intervalo

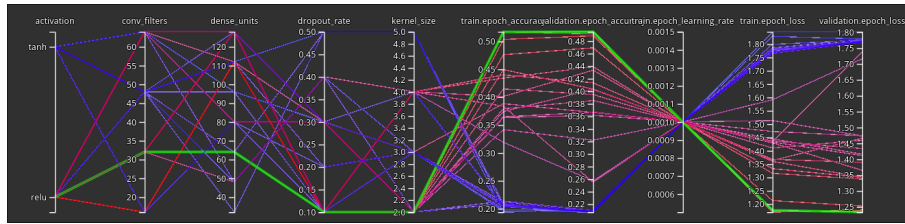


Figura C.6: Gráfico de entrenamiento del modelo CNN con 0.4s de intervalo (mejor val_accuracy = 0.49473)

C.3. Intervalo 0.6s

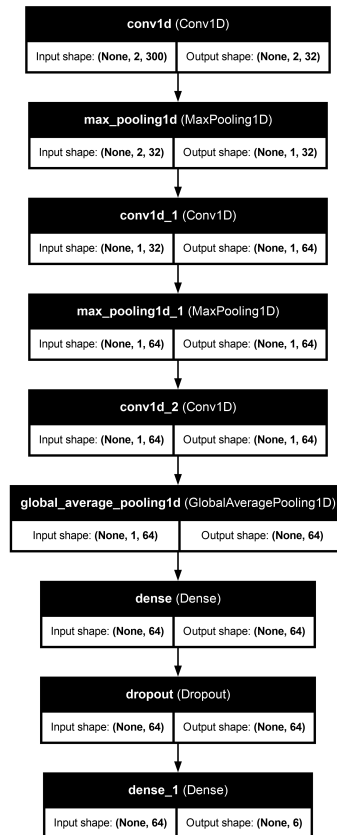


Figura C.7: Esquema del modelo CNN con 0.6s de intervalo

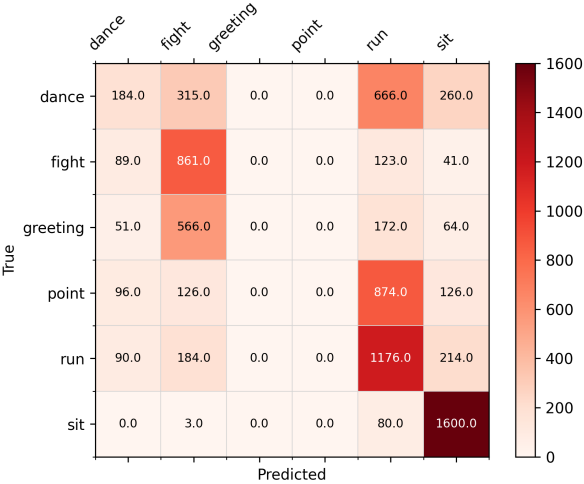


Figura C.8: Matriz de confusión del modelo CNN con 0.6s de intervalo

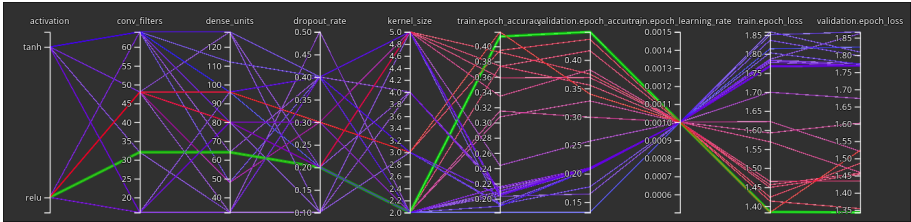


Figura C.9: Gráfico de entrenamiento del modelo CNN con 0.6s de intervalo (mejor val_accuracy = 0.44952)

C.4. Intervalo 0.8s

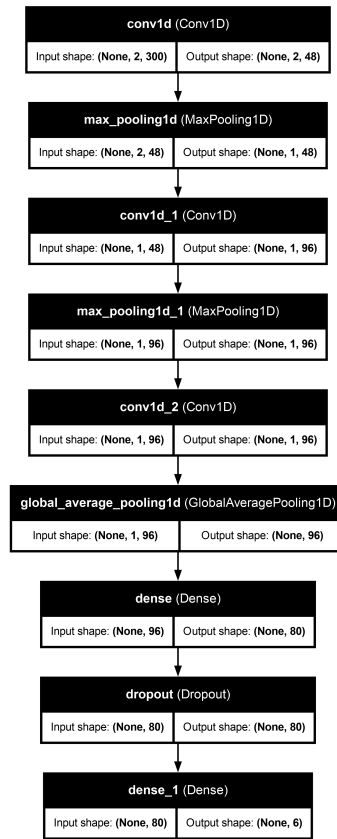


Figura C.10: Esquema del modelo CNN con 0.8s de intervalo

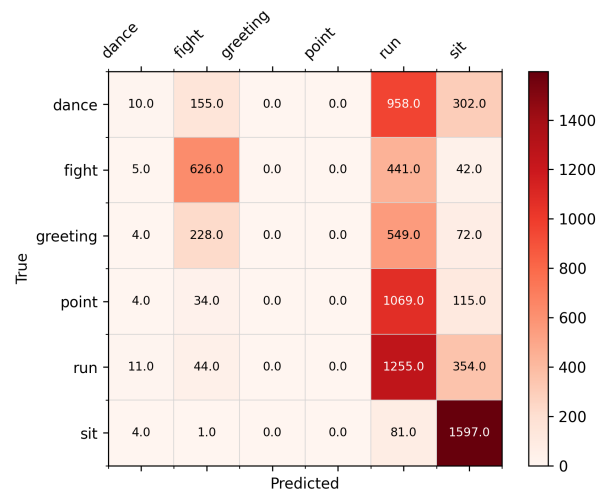


Figura C.11: Matriz de confusión del modelo CNN con 0.8s de intervalo

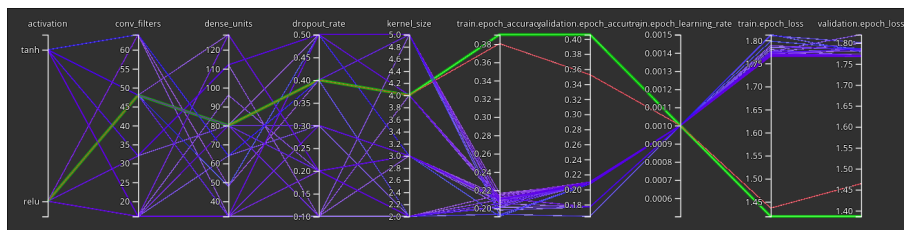


Figura C.12: Gráfico de entrenamiento del modelo CNN con 0.8s de intervalo (mejor val_accuracy = 0.40583)

C.5. Intervalo 1.0s

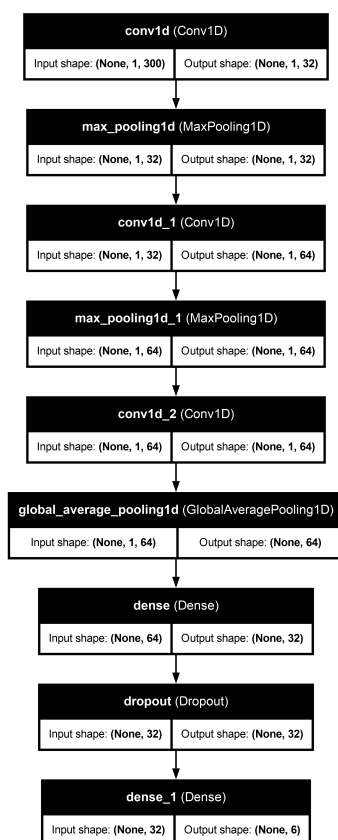
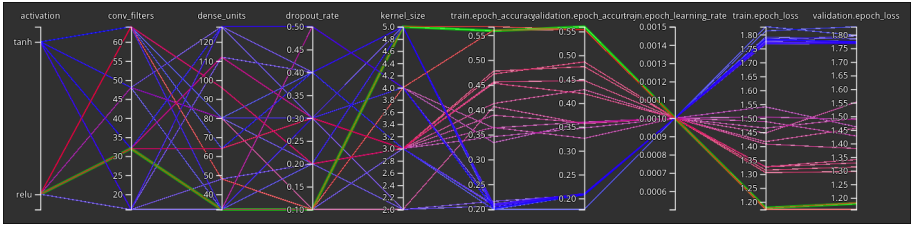


Figura C.13: Esquema del modelo CNN con 1.0s de intervalo



Figura C.14: Matriz de confusión del modelo CNN con 1.0s de intervalo



Glosario

CNN Convolutional Neural Network

IA Inteligencia Artificial

VR Realidad Virtual

Licencia de uso del documento

Esta obra está bajo una licencia [Creative Commons](http://creativecommons.org/licenses/by-sa/4.0/legalcode) «Atribución-NoComercial-CompartirIgual 4.0 Internacional».



<http://creativecommons.org/licenses/by-sa/4.0/legalcode>.

Licencia de uso del código fuente

Los archivos de código fuente para generar este documento se encuentran en <https://github.com/FratosVR/Memoria-TFG> bajo la licencia [GPL-3.0](#)